

100071011 Computer Networks 2025-2026-2
Project-1
Reliable File Transfer using Go-Back-N protocol
Project Report

学号 (Student ID)	1120231863
姓名 (Name)	左逸龙
班号 (Class No.)	07112303
授课教师 (Instructor)	宿红毅

School of Computer
Beijing Institute of Technology
May 19, 2026

1. Requirement Analysis

This project requires a reliable file transfer program based on UDP sockets. UDP only provides datagram delivery and does not guarantee reliable transmission. Therefore, reliability mechanisms such as checksum verification, acknowledgement, timeout, and retransmission need to be implemented at the application layer.

The main requirements and their corresponding implementation positions are listed in Table 1.

No.	Requirement from the project description	How it is handled	Evidence in source project
1	Define a PDU and add CRC-CCITT checksum.	The program defines a fixed PDU header and calculates CRC when encoding/decoding.	gbn/pdu.py
2	Use UDP Socket API; one datagram contains one PDU.	Each encoded PDU is sent by UDP sendto and read by recvfrom.	gbn/host.py
3	Data field should not be larger than 4 KB.	The host checks data_size, and PDU encoding also checks the payload length.	gbn/pdu.py, gbn/host.py
4	Use Go-Back-N for reliable transfer.	The sender keeps a window and retransmits unacknowledged PDUs after timeout.	gbn/host.py
5	Support full-duplex transfer.	Each host has a receive loop and sender threads, so Host1 and Host2 can send at the same time.	run_demo.py, gbn/host.py
6	Simulate PDU loss and PDU error by configured rates.	The channel wrapper can drop or corrupt outgoing DATA/FIN PDUs.	gbn/channel.py, configs/host1.json
7	Use files larger than 3 MB for testing.	The generator creates binary test files in data/.	generate_test_file.py
8	Received file should be the same as the original file.	The demo writes received files into received/ for later checking.	run_demo.py
9	Use configuration files.	Host parameters and peer information are stored in JSON files.	configs/*.json
10	Record communication status and analyze logs.	CSV logs are generated and then summarized by an analysis script.	gbn/logging_utils.py, analyze_logs.py

Table 1: Main project requirements used in this implementation.

The expected output of the system includes received files, communication records, and analysis results generated from these records.

2. Design

2.1. System Model

The main data path is shown in Figure 1. For one direction, the sender reads the file, creates GBN DATA PDUs, adds CRC, and sends them through UDP. The receiver checks the PDU and writes accepted data into the output file. The same structure is also used in the other direction, so the program can do full-duplex transfer.

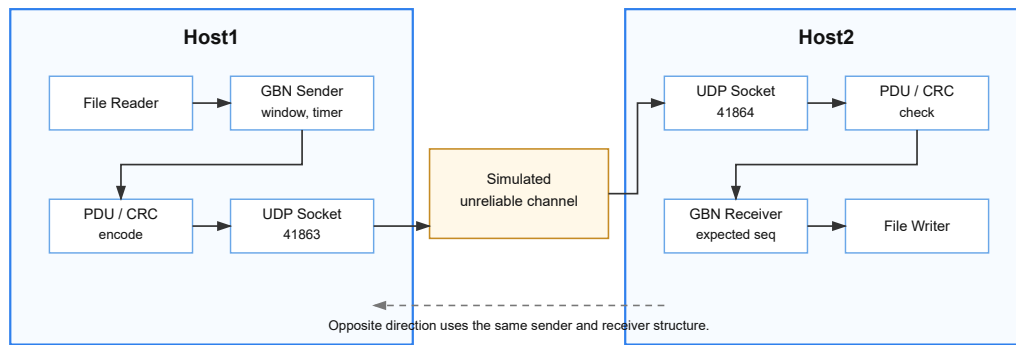


Figure 1: Main data path of the file transfer system.

2.2. PDU Format

The PDU fields are listed in Table 2. We used a 16-byte fixed header and a variable data field. The data field is limited to 4096 bytes.

Field	Size	Meaning
Magic	2 bytes	Protocol identifier, value 0x4742.
Version	1 byte	Protocol version, currently 1.
Type	1 byte	DATA, ACK, FIN, or FIN_ACK.
Session ID	4 bytes	Identifier of one file transfer session.
Seq	2 bytes	Sequence number in the configured sequence space.
Ack	2 bytes	Acknowledgement number.
Length	2 bytes	Length of the data field.
Checksum	2 bytes	CRC-CCITT-FALSE checksum.
Data	0-4096 bytes	File payload for DATA PDUs.

Table 2: PDU format used by the program.

When a PDU is encoded, the checksum field is first set to zero. Then the CRC value is calculated over the header and data. When a PDU is received, the same calculation is done again. If the value is different, the PDU is treated as damaged.

2.3. Go-Back-N Protocol

The sender keeps all sent but unacknowledged PDUs in the current window. In the code, `base` is the first unacknowledged frame and `next_abs` is the next frame to send. ACKs are cumulative. When the sender receives a useful ACK, `base` moves forward.

A simple timeout example is shown in Figure 2. If a DATA PDU is lost or damaged, later DATA PDUs are discarded by the receiver because it is still waiting for the missing sequence number. When the sender timer expires, the sender retransmits the outstanding PDUs from the first unacknowledged one.

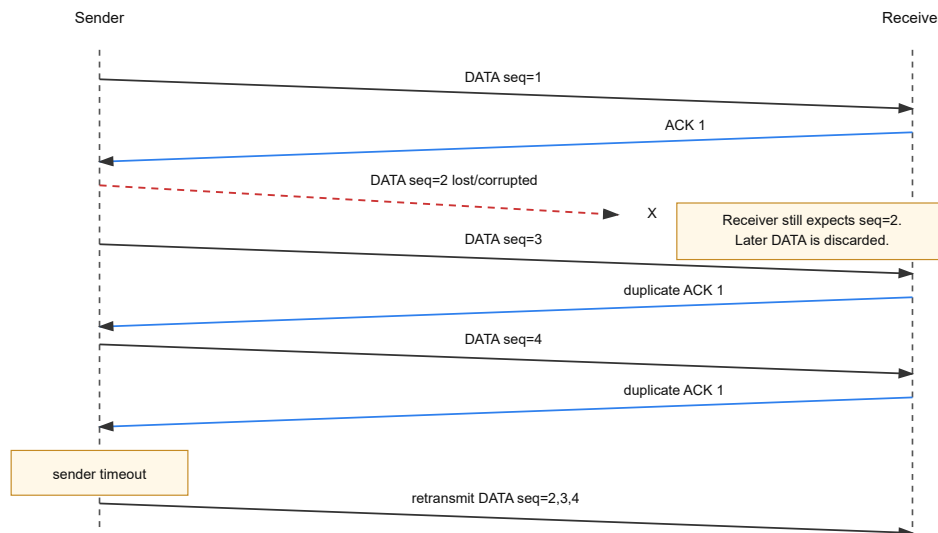


Figure 2: A simple Go-Back-N timeout and retransmission example.

The receiver rule is summarized in Figure 3. The receiver window size is 1. It accepts only the expected sequence number and does not save out-of-order DATA PDUs.

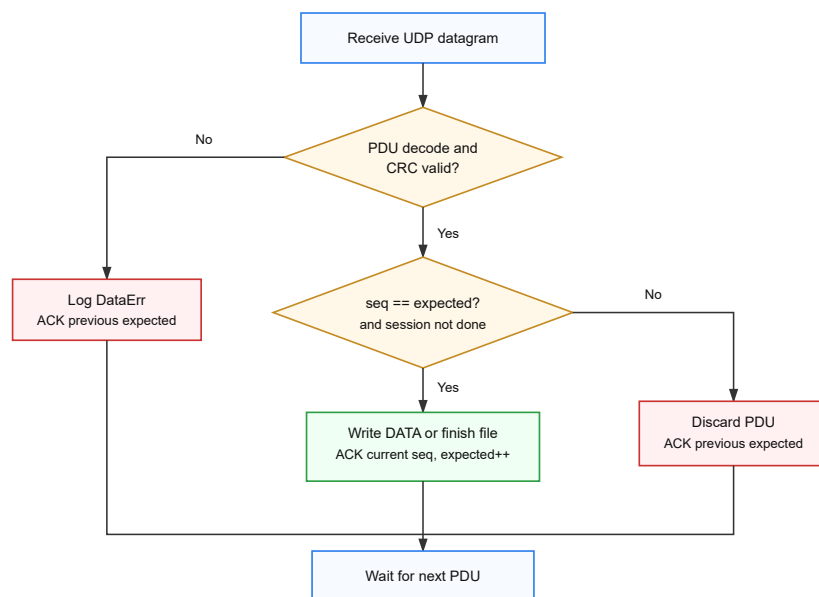


Figure 3: Receiver logic with receive window size 1.

The sequence number written into the PDU is a modular number. The program still uses absolute frame numbers internally. During startup, it checks $sw_size \leq 2^{seq_bits} - 1$.

2.4. Configuration and Error Simulation

The main configurable parameters are shown in Table 3. They are stored in JSON files under configs/.

Parameter	Default value	Meaning
data_size	1024	Bytes of file data in one DATA PDU.
seq_bits	8	Sequence number space is 2^8 .
sw_size	8	Sending window size.
init_seq_no	1	Initial sequence number used by both sides.
timeout_ms	300	Timeout value in milliseconds.
lost_rate	0	Random DATA/FIN loss percentage.
error_rate	0	Random DATA/FIN corruption percentage.
local_port	41863/41864	UDP ports used by Host1 and Host2.

Table 3: Important configuration parameters.

In this implementation, random loss and corruption are applied to DATA and FIN packets. ACK and FIN_ACK packets are not randomly damaged or dropped. Also, InitSeqNo is read from the configuration file, so both sides should use the same value.

3. Development and Implementation

The program was developed and tested on Windows. PowerShell was used to run the commands. Python was used as the programming language, and uv was used to create the virtual environment.

The important files in the source project are shown below.

```

source project/
├── gbn/                                // main protocol package
│   ├── pdu.py                        // PDU format, CRC, and sequence helpers
│   ├── host.py                      // sender, receiver, ACK, timeout, file writing
│   ├── channel.py                   // random loss and corruption simulation
│   ├── logging_utils.py             // CSV communication records
│   └── analyzer.py                 // statistics from communication records
├── configs/                          // JSON configuration files
│   ├── host1.json                  // Host1 parameters and peers
│   ├── host2.json                  // Host2 parameters and peers
│   ├── host3.json                  // optional multi-host configuration
│   └── host4.json                  // optional multi-host configuration
├── tests/
│   └── test_pdu.py                 // unit tests for PDU and sequence helpers
├── run_demo.py                      // local full-duplex demo for Host1 and Host2
├── run_host.py                     // start one host from one configuration file
├── analyze_logs.py                 // run log analysis
└── generate_test_file.py           // generate binary test files

```

The CRC code is in gbn/pdu.py. We used the standard CRC-CCITT-FALSE initial value and polynomial.

```

def crc_ccitt_false(data: bytes) -> int:
    crc = 0xFFFF
    for byte in data:
        crc ^= byte << 8
        for _ in range(8):
            if crc & 0x8000:
                crc = ((crc << 1) ^ 0x1021) & 0xFFFF
            else:
                crc = (crc << 1) & 0xFFFF
    return crc

```

The timeout part is in gbn/host.py. When the timer expires, the sender sends the current outstanding window again.

```

if timer_started is not None and time.monotonic() - timer_started >= self.timeout:
    log.log(direction="send", event="timeout", status="T0", base=base,
    next_seq=next_abs)
    for abs_no in range(base, next_abs):
        self._send_pdu(frames[abs_no], address, log, "T0", base, next_abs)
    timer_started = time.monotonic()

```

4. System Deployment, Startup, and Use

The following commands are written for PowerShell. First create and activate the uv environment in the source project directory.

```

uv venv --prompt "Project-1"
.\\.venv\\Scripts\\Activate.ps1

```

Then generate two test files. Each generated file is larger than 3 MB.

```

python generate_test_file.py data/host1.bin
python generate_test_file.py data/host2.bin --seed 202

```

The key fields in configs/host1.json are shown below.

```

{
  "host_id": "Host1",
  "local_ip": "127.0.0.1",
  "local_port": 41863,
  "data_size": 1024,
  "seq_bits": 8,
  "sw_size": 8,
  "init_seq_no": 1,
  "timeout_ms": 300,
  "error_rate": 0,
  "lost_rate": 0,
  "peers": [
    {
      "peer_id": "Host2",
      "peer_ip": "127.0.0.1",
      "peer_port": 41864,
      "send_file": "data/host1.bin",
      "receive_file": "received/from_host2.bin"
    }
  ]
}

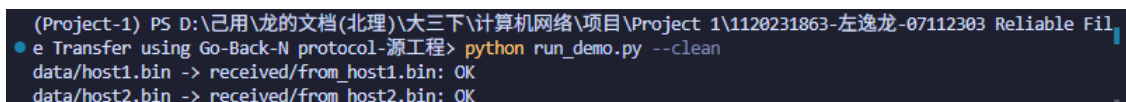
```

For the local demo, run:

```

python run_demo.py --clean
python analyze_logs.py --log-dir logs --output-dir results

```



```

(Project-1) PS D:\已用\龙的文档(北理)\大三下\计算机网络\项目\Project 1\1120231863-左逸龙-07112303 Reliable File Transfer using Go-Back-N protocol-源工程> python run_demo.py --clean
data/host1.bin -> received/from_host1.bin: OK
data/host2.bin -> received/from_host2.bin: OK

```

Figure 4: PowerShell output of the local full-duplex demo.

The hosts can also be started manually in two terminals:

```

# Terminal 1
python run_host.py configs/host1.json

```

```
# Terminal 2
python run_host.py configs/host2.json
```

5. System Test

5.1. Unit Tests

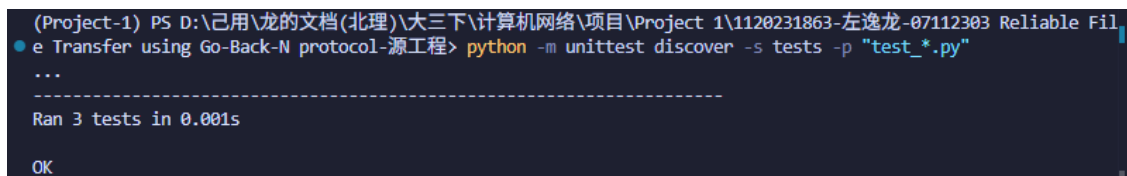
The automatic unit tests are in `tests/test_pdu.py`. There are three test methods. The main test code is shown below.

```
class PduTests(unittest.TestCase):
    def test_crc_standard_vector(self):
        self.assertEqual(crc_ccitt_false(b"123456789"), 0x29B1)

    def test_pdu_round_trip_and_crc_failure(self):
        pdu = PDU(TYPE_DATA, 1234, 7, 6, b"hello")
        encoded = bytearray(pdu.encode())
        decoded = PDU.decode(bytes(encoded))
        self.assertEqual(decoded, pdu)
        encoded[-1] ^= 0x55
        with self.assertRaises(PDUError):
            PDU.decode(bytes(encoded))

    def test_sequence_wrap_window(self):
        self.assertEqual(seq_mod(258, 8), 2)
        self.assertTrue(in_window(250, 250, 12, 8))
        self.assertTrue(in_window(5, 250, 12, 8))
        self.assertFalse(in_window(6, 250, 12, 8))
```

The unit tests were run in PowerShell, and the result is shown in Figure 5.



```
(Project-1) PS D:\已用\龙的文档(北理)\大三下\计算机网络\项目\Project 1\1120231863-左逸龙-07112303 Reliable File Transfer using Go-Back-N protocol-源工程> python -m unittest discover -s tests -p "test_*.py"
...
-----
Ran 3 tests in 0.001s
OK
```

Figure 5: PowerShell output of the unit tests.

5.2. Integrated Checks

The other checks were not all written as unit tests. They were run as integration or stress checks by changing the configuration and then running the transfer program. This is summarized in Table 4.

Test item	How it was tested	Expected result	Result
Unit tests	Run tests/test_pdu.py through unittest.	All tests pass.	Passed
Full-duplex transfer	Run Host1 and Host2 demo.	Both directions finish.	Passed
File equality	Compare original and received files by SHA-256.	Hashes are the same.	Passed
Packet loss	Set lost_rate=5, error_rate=0.	Timeout and retransmission occur.	Passed
Packet corruption	Set lost_rate=0, error_rate=5.	CRC errors are recorded.	Passed
Loss and corruption	Set both rates to 5%.	Transfer still completes.	Passed
Maximum data size	Set data_size=4096.	Program can run.	Passed
Invalid data size	Set data_size=4097.	Program rejects it.	Passed
Sequence wrap-around	Set seq_bits=3, sw_size=7.	Transfer still works.	Passed
Invalid window size	Set seq_bits=3, sw_size=8.	Program rejects it.	Passed
Multi-host transfer	Run Host1 sending to Host2, Host3, and Host4.	All received files match.	Passed

Table 4: Main test and checking items.

For file equality, we used SHA-256. If the two hashes are the same, the received file is considered identical to the original file. This check is important because a transferred binary file may look successful from the terminal output, but still be wrong by a few bytes. The PowerShell hash result is shown in Figure 6.

```
(Project-1) PS D:\己用\龙的文档(北理)\大三下\计算机网络\项目\Project 1\1120231863-左逸龙-07112303 Reliable File Transfer using Go-Back-N protocol-源工程> Get-FileHash data\host1.bin -Algorithm SHA256
>> Get-FileHash received\from_host1.bin -Algorithm SHA256
>> Get-FileHash data\host2.bin -Algorithm SHA256
● >> Get-FileHash received\from_host2.bin -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	DCD36F1582DD0B72F11130426A20BAA6F31C7F1A742DCA07B6021A5215512939	D:\己用\龙的文档(北理...
SHA256	DCD36F1582DD0B72F11130426A20BAA6F31C7F1A742DCA07B6021A5215512939	D:\己用\龙的文档(北理...
SHA256	35DD3C26D3239F0891EFC44B39632CD01CE45CA36022189519B200D4705132B5	D:\己用\龙的文档(北理...
SHA256	35DD3C26D3239F0891EFC44B39632CD01CE45CA36022189519B200D4705132B5	D:\己用\龙的文档(北理...

Figure 6: PowerShell output of SHA-256 file hash comparison.

The sender log also shows Go-Back-N behavior. In one default run, the sender first sent DATA 1 to DATA 8. Then a timeout occurred and the sender retransmitted the outstanding window. A short excerpt is shown in Table 5.

Event	Type	Seq	Status	Base	Next
pdu	DATA	1	New	0	1
pdu	DATA	2	New	0	2
pdu	DATA	8	New	0	8
timeout			T0	0	8
pdu	DATA	1	T0	0	8
pdu	DATA	2	T0	0	8
pdu	DATA	3	T0	0	8

Table 5: Short sender log excerpt showing timeout retransmission.

6. Performance and Analysis

The analysis result is used to compare different loss and error settings. The receiver-side throughput of the default demo is shown in Figure 7, and the retransmission count is shown in Figure 8.

Receiver log	Duration	Bytes OK	Throughput
Host1_recv_Host2_ec50f047.csv	17.046089 s	3145851	180.22 KiB/s
Host2_recv_Host1_f4dd0758.csv	16.816290 s	3145851	182.69 KiB/s

Table 6: Receiver-side statistics in the default demo.



Figure 7: Receiver-side throughput in the default full-duplex demo.

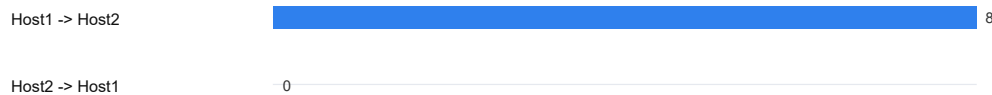


Figure 8: Retransmitted PDU count in the default full-duplex demo.

From the results, packet loss mainly increases timeout and retransmission. Packet corruption is shown by DataErr records. The stress-check results are listed in Table 7.

Scenario	Loss	Error	Retransmits	Timeouts	DataErr
Loss only	5%	0%	2463	308	0
Corruption only	0%	5%	-	-	430
Loss and corruption	5%	5%	5440	680	556

Table 7: Effect of packet loss and packet corruption.

These results match the behavior of Go-Back-N. Once one PDU is missed, the receiver does not keep later PDUs, so the sender may need to send several PDUs again.

7. Summary or Conclusions

In this project, we implemented a reliable file transfer program over UDP. The main work was to define the PDU, add CRC checking, implement the Go-Back-N sender and receiver, handle timeout retransmission, and use configuration files to run different cases.

The program can transfer files larger than 3 MB in both directions. The received files were checked by SHA-256 in the test stage. Loss and corruption tests also showed that the program can recover by retransmission.

There are still some simple limitations. The initial sequence number should be set consistently on both hosts. Also, ACK and FIN_ACK packets are not randomly damaged or dropped in this implementation. A possible improvement is to add a small handshake to negotiate initial parameters.

8. References

1. Andrew S. Tanenbaum and David J. Wetherall, *Computer Networks*, 5th Edition, Prentice Hall, 2011.
2. J. Postel, "User Datagram Protocol", RFC 768, 1980.
3. J. Postel, "Transmission Control Protocol", RFC 793, 1981.

4. R. Braden, D. Borman, and C. Partridge, “Computing the Internet Checksum”, RFC 1071, 1988.
5. RevEng CRC Catalogue, “CRC-16/IBM-3740”, which gives the CRC-CCITT-FALSE parameter set.
6. Course project handout, “Network Programming Project-1 Reliable file transfer using Go-Back-N protocol”.

9. Comments

This project is useful for understanding the difference between UDP and reliable transport. The project description could give a clearer minimum log format and also state whether ACK packets should be included in the random loss/error simulation. It would also be helpful to specify how many performance comparison cases are expected in the report.